

B 2.2.3 Stacks Python



B 2.2.3 Explain the concept of a stack as a “last in, first out” (LIFO) data structure Python

This section explores the stack data structure, which operates on the LIFO principle (like a stack of plates!). We'll cover key operations (`push`, `pop`, `peek`, `isEmpty`), their efficiency, and how to choose the right stack implementation (array-based or linked list-based) for your needs.

Key Questions

- How does a stack operate as a "last in, first out" (LIFO) data structure?
- What are the key stack operations such as push, pop, peek, and isEmpty, and how do these affect performance and memory usage?
- How do you determine an appropriate stack for a specific problem?

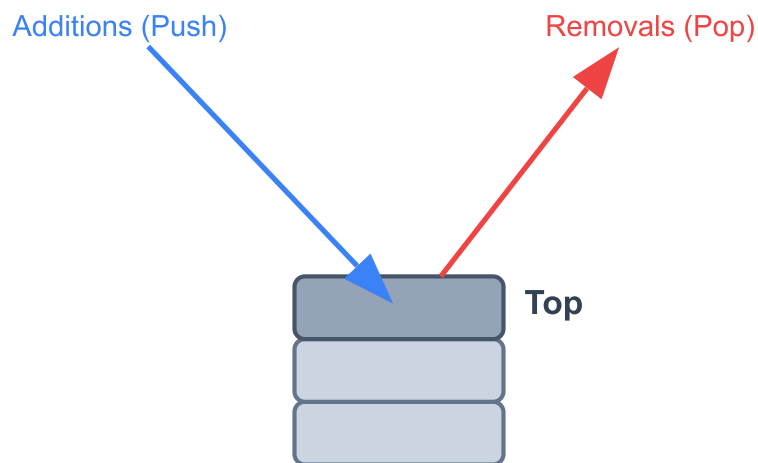
What is a Stack?

A stack is a fundamental linear data structure in computer science. Its defining principle is **LIFO**, which stands for "**Last-In, First-Out.**"

This means that the last item you add to the stack is always the first item you can remove.

A simple way to visualise this is to think of a stack of plates or a pile of books. You add new plates to the top of the stack, and when you need a plate, you remove the one from the top. You can't easily remove a plate from the middle or the bottom without taking the top ones off first.

Stack: LIFO Structure



Fundamental Stack Operations

There are four primary operations you must know for managing a stack.

1. push(item)

- **What it does:** Adds a new element (an **item**) to the very top of the stack.
- **Error Check:** If the stack is implemented with a fixed size (like an array) and is already full, attempting to **push** a new item will cause a **Stack Overflow** error.

2. pop()

- **What it does:** Removes the element from the top of the stack and returns that element. The stack's size decreases by one.
- **Error Check:** If you try to `pop` an item from an empty stack, it will cause a **Stack Underflow** error, as there is nothing to remove.

3. peek()

- **What it does:** Returns the element at the top of the stack *without* removing it. This is useful for checking what the next item to be popped is.
- **Error Check:** Just like `pop`, attempting to `peek` at an empty stack will cause a **Stack Underflow** error.

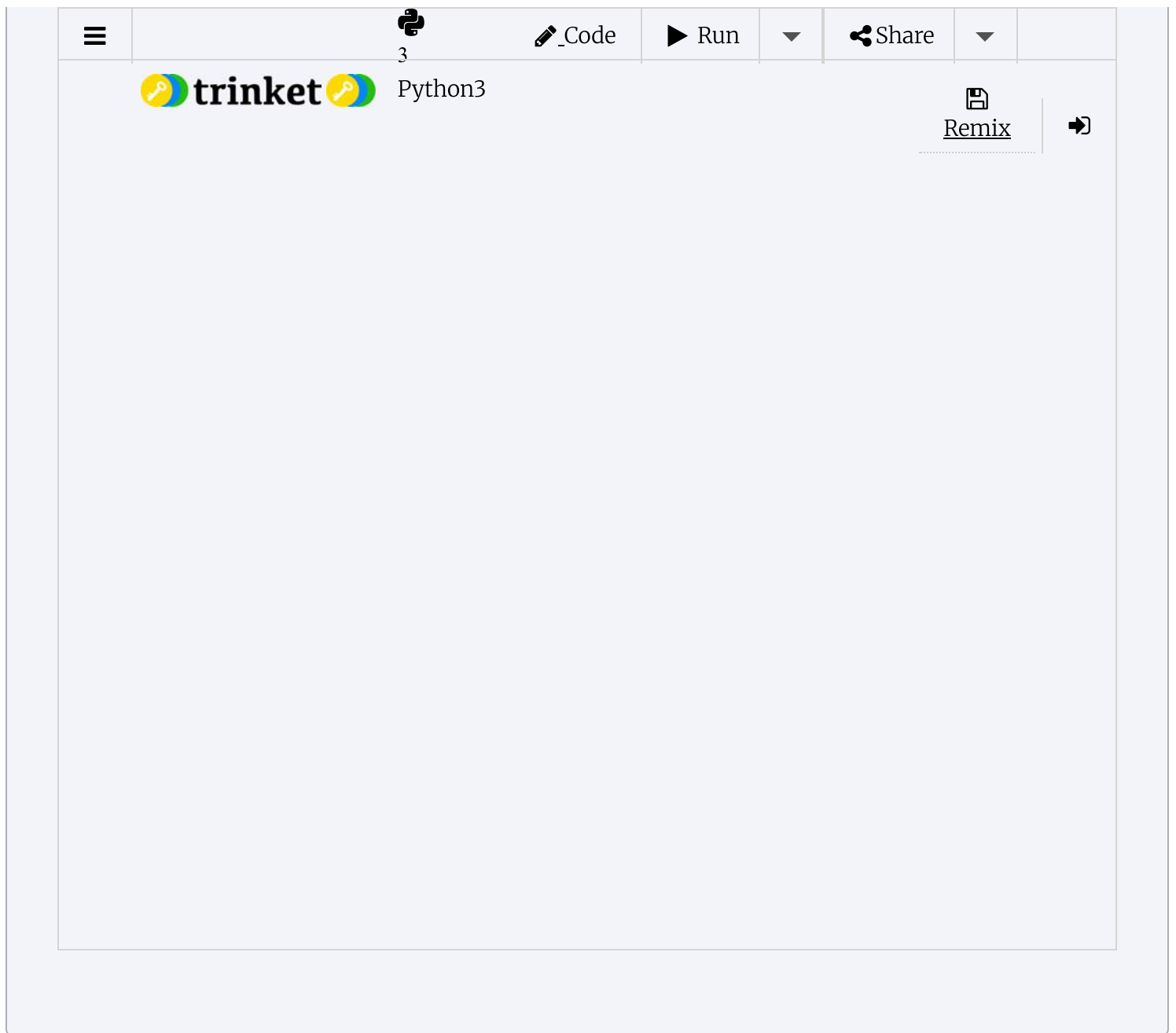
4. isEmpty()

- **What it does:** A utility operation that checks if the stack has any elements in it. It returns a boolean value (`true` if the stack is empty, `false` otherwise).
- **Why it's important:** You should always use `isEmpty()` to check the state of a stack before calling `pop()` or `peek()` to prevent underflow errors.

Interactive Practice: Array-Based Stack

Use the code editor below to practice. A basic stack has been implemented using an array in the `Stack.java` file.

- Try using `push()` to add items to the stack.
- Use `pop()` to remove them and see the LIFO principle in action.
- Use `peek()` to check the top item without removing it.
- Try to `pop` from an empty stack to see the underflow error message.



Performance and Memory Usage

Stack operations (push, pop, peek, isEmpty) are typically very efficient and have a time complexity of $O(1)$, meaning they take a constant amount of time regardless of the stack size. Memory usage grows linearly with the number of elements in the stack.

Choosing the Right Stack

The choice of stack implementation often depends on the specific needs of your program:

- Array-based stacks: Simple to implement, good for fixed-size stacks.
- Linked list-based stacks: More flexible for dynamic resizing.

When making your choice, consider factors like the expected size of the stack, frequency of operations, and memory constraints.

Python Implementation

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        if not self.is_empty():
            return self.items.pop()
    def peek(self):
        if not self.is_empty():
            return self.items[-1]
    def is_empty(self):
        return len(self.items) == 0

# Example usage
my_stack = Stack()
my_stack.push(10)
my_stack.push(20)
my_stack.push(30)
print(my_stack.pop()) # Output: 30
print(my_stack.peek()) # Output: 20
```

1

Which of the following describes the LIFO principle in a stack?

A. ☐ First In, Last Out

B. ☒ Last In, First Out

C. ☐ Last In, Last Out

D. ☐ First In, First Out

2

What does the push() operation do in a stack?

A. ☐ Removes the top element

B. ☐ Checks if the stack is empty

C. ☐ Returns the top element without removing it

D. ☒ Adds an element to the top

3

What will the output of this code be?

```
my_stack = Stack()  
my_stack.push(5)  
my_stack.push(10)  
print(my_stack.pop())
```

A. ☒ 10

B. ☐ Error

C. ☐ 5

D. ☐ 15

4

Which method allows you to view the top element of a stack without removing it?

A. ☒ peek()

- B. ☐ push()
- C. ☐ pop()
- D. ☐ is_empty()

5

What is the time complexity of the pop() operation in a well-implemented stack?

- A. ☐ $O(n)$
- B. ☐ $O(\log n)$
- C. ☐ $O(n^2)$
- D. ☒ $O(1)$

6

What is the primary advantage of using a linked list-based stack over an array-based stack?

- A. ☒ Dynamic resizing
- B. ☐ Faster access to elements
- C. ☐ Lower memory usage
- D. ☐ Easier implementation

7

If you need to implement a stack with a fixed maximum size, which implementation would be more suitable?

- A. ☐ Both are equally suitable

B. ☒ Array-based stack

C. ☐ Linked list-based stack

D. ☐ Neither is suitable

8

In the Python code example, what does `self.items[-1]` do?

A. ☐ Removes the last element from the list

B. ☐ Adds an element to the beginning of the list

C. ☐ Checks if the list is empty

D. ☒ Accesses the last element of the list

9

What does the `is_empty()` method do?

A. ☒ Checks if the stack has no elements

B. ☐ Returns the size of the stack

C. ☐ Adds a default element to the stack

D. ☐ Removes all elements from the stack

10

Which real-world scenario can be effectively modeled using a stack?

A. ☐ Representing a family tree

B. ☒ Tracking the history of visited web pages

- C. ☐ Storing a list of contacts
- D. ☐ Managing a queue of customers

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**